

# CSC 330 Programming Languages

## Program #3

### Custom Parser

#### Directions:

Recall the grammar from homework 1. In that homework, you manually figured out the parse tree for a given expression. Now we are going to use ANTLR to parse the expression and interpret it. In fact, it will interpret a file with multiple expressions in it.

#### Installation:

To quickly test your lexer and parser rules, you can use <http://labantlr.org>. You write the lexer tokenization rules under "Lexer" and the parser rules under "Parser". It even gives you the abstract syntax tree for the given input when you hit Run.

To actually generate code, you will need to create a .g4 file (for ANTLR4 grammar) and install the ANTLR tools. Go to <https://www.antlr.org/index.html> and follow directions for Quick Start. This will allow you to use the command line ANTLR tool to generate Java code from your .g4 file. The .g4 file will include both your parser and your lexer rules. Note that the "antlr4-parse" command sometimes doesn't work correctly, however this command isn't strictly needed. If you want to see the abstract syntax tree, use the lab.antlr.org online tool. The only tool you have to have working is the "antlr4" one, which generates the code.

Once you have generated files (there are 4 of them, all ending in .java), you should be able to put them next to the provided WeirdCalc.java file and open Git Bash in that directory. To run everything, you will need the antlr.jar file. You can download it here: <https://www.antlr.org/download.html> scroll down to "ANTLR tool and Java Target" and download the "ANTLR 4.13.1 Java runtime binaries jar". This will give you a single jar file named "antlr-runtime-4.13.1.jar". I recommend renaming this to just "antlr.jar" so there's less typing. Put this jar file next to WeirdCalc.java and the generated .java files. In Git Bash, type the following command:

```
export CLASSPATH=".;antlr.jar"
```

this assumes you renamed the jar to "antlr.jar". This command will allow the java compiler and the java runtime environment to use this jar. Now just compile all java files (i.e. "javac \*.java" in the terminal) and then run the application (i.e. "java WeirdCalc").

#### Running the Program:

By default the WeirdCalc class reads from the file "source.wc" (where .wc is our file extension which stands for "weird calculator"). This file is given to you but it contains a lot of lines of code. It is very unlikely that you will run all lines successfully the first time you try this. Therefore it is HIGHLY RECOMMENDED that you create your own .wc file and put your own much simpler code inside of it. You can then specify this input file in the terminal when you run the program. Simply put the name of the input file at the end, e.g.

```
java WeirdCalc myfile.wc
```

#### Error Correction:

ANTLR has built in error reporting. It should report the details of all syntax errors to the console. In addition to this, you will also need to report the use of any uninitialized variables (refer to the grammar on the next page to see that there are assignment statements now). Just like in other languages, a variable must be given a value before we can read from it. This error must be printed to System.err since it is, in fact, an error. System.out should be reserved for normal messages while System.err is for errors.

## Grammar:

Now that we have our tools in place, we will need to create both the lexer and parser rules. For each terminal token, you will need to use regular expressions to create the lexer tokenization rules. For each grammar rule, you will need to convert these to ANTLR's syntax in the parser section. ANTLR allows you to insert custom code to be executed each time a production rule is matched. The grammar we will be using is a slightly modified form of the grammar from homework 1. This is the full grammar we are using (in BNF format):

```
<program> ::= <program> <line> | ""
<line> ::= <assignment> ";" | <print> ";"
<assignment> ::= <identifier> "=" <exp>
<print> ::= "print" "(" <exp> ")" | "print" "(" " " ")"
<exp> ::= <exp> "#" <item> | <item>
<item> ::= <operand> "@" <item> | <operand> "~" <item> | <operand>
<operand> ::= "$" <operand> | <data>
<data> ::= <number> | <identifier>
<number> ::= "-" <digits> | <digits>
<digits> ::= "0"..."9" <digits> | "0"..."9"
<identifier> ::= "a"..."z" <identifier_rest> | "A"..."Z" <identifier_rest>
<identifier_rest> ::= "a"..."z" <identifier_rest>
                        | "A"..."Z" <identifier_rest>
                        | "0"..."9" <identifier_rest>
```

All terminal tokens should be handled by the lexer using regular expressions. For the above grammar, this will include ";", "=", "print", "(", ")", "#", "@", "~", "\$", "-". We should also do this for the non-terminals <digits> and <identifier>, since these would be better handled by the lexer. You will then replace the literals (as well as <digits> and <identifier>) with the corresponding tokens. For example, instead of:

```
"print" "(" <exp> ")"
```

it would be:

```
PRINT LPAREN <exp> RPAREN
```

where PRINT is the string literal "print" in the lexer, LPAREN is "(", and RPAREN is ")".

## Interpreter:

To do this correctly, you will need to use the dictionary in the WeirdCalc class. Use it in the grammar file. The key will be the variable's name and the value will be the value currently stored in that variable. This should be instantiated when the `program` non-terminal starts to match (look up how to do this in ANTLR using the `@init` keyword). Every time an assignment expression is found, you will need to add or update its entry in the dictionary. Every time a variable is referenced, you will need to retrieve its current value from the dictionary. You will also need to carry out all the math inherent in the unusual arithmetic symbols. As a reminder, here is the explanation of those symbols:

`X # Y` evaluates to 0 if `X mod Y` is 0, otherwise it evaluates to 1

`$X` evaluates to -1 if `X` is negative, 0 if `X` is zero, +1 if `X` is positive

`X @ Y` adds `X` and `Y` together and takes the absolute value of their sum, i.e.  $|X + Y|$

`X ~ Y` squares `X` and `Y` individually then adds the squares, i.e.  $X^2 + Y^2$

`-X` is normal unary minus

For print statements, you will need to use the expression's value and print that out to standard output in the terminal. For the rule where print does not have an expression, you will need to just go to the next line (without actually printing anything), such that the next print is on the following line in the terminal.

### BONUS:

For extra points, create a program to apply syntax highlighting to our source.wc file.

### Overview

You will create a Java app that applies syntax highlighting to a source code wc file. The desired output will look like the following:

```
$ java Highlight source theme1
// from homework
print($2 ~ -3 @ -1 # 3); // should print 1
print(--5 @ $4);         // should show: line 3:7 no viable alternative at input '--'
                        // may print a 0 after the error (if so, ignore it)
print(34 ~ $8);          // should print 1157
print(8 - 4 # 4 # 5);    // should show: line 5:8 mismatched input '-' expecting {'}', '#'}
print($-3 @ 0 # $1);     // should print 0
print();

// simple variable test
counter = 0;             // should assign 0 to counter
print(counter);          // should print 0
counter = counter @ 1;   // should assign 1 to counter
print(counter);          // should print 1
print();

// more complicated variable test
number = -5;             // should assign -5 to number
print(number);           // should print -5
print($number);          // should print -1
number = number ~ number; // should assign 50 to number
print(number);           // should print 50
print(number # 5);        // should print 0
print(number # 4);        // should print 1
print(number @ -125);     // should print 75
```

By running the app and giving it the name of the source code file (this time without the extension) and the name of one of our color themes, it will print out the contents of the source code file using the specified color profile.

### Process

Create a new folder called “bonus”. Copy your WeirdCalc.java, WeirdCalc.g4, antlr.jar, and source.wc files into that folder. Rename WeirdCalc.java to Highlight.java (also rename the class). First objective is to get the Java app to just print out the contents of the source.wc file after running the lexer. So we will need to iterate over the tokens and print out the matched text. We won’t need the parser at all for this. You will have to modify the grammar and the Highlight.java code to get this to work. This will help you:

<https://www.antlr.org/api/Java/org/antlr/v4/runtime/CommonTokenStream.html> . Since we won’t need the parser, you should have the parser part of the grammar file just be an empty starting rule and that’s it. Remove all parts of the grammar and Highlight.java file that we no longer need.

```

$ java Highlight source
// from homework
print($2 ~ -3 @ -1 # 3); // should print 1
print(--5 @ $4);         // should show: line 3:7 no viable alternative at input '--'
                        // may print a 0 after the error (if so, ignore it)
print(34 ~ $8);           // should print 1157
print(8 - 4 # 4 # 5);     // should show: line 5:8 mismatched input '-' expecting {'}', '#'}
print($-3 @ 0 # $1);     // should print 0
print();

// simple variable test
counter = 0;             // should assign 0 to counter
print(counter);          // should print 0
counter = counter @ 1;   // should assign 1 to counter
print(counter);          // should print 1
print();

// more complicated variable test
number = -5;             // should assign -5 to number
print(number);           // should print -5
print($number);          // should print -1
number = number ~ number; // should assign 50 to number
print(number);           // should print 50
print(number # 5);        // should print 0
print(number # 4);        // should print 1
print(number @ -125);     // should print 75

```

Now we need to create a color theme. This can be accomplished with a mapping from token type to color. First, create a hard coded map in Java that maps token type (which is an integer) to ansi color code (which is also an integer). When you go through the token stream to print out the tokens, you should look up the token type in your color map and set the terminal color appropriately, then print the token's text. We will be using these colors:

| print this | for this color |
|------------|----------------|
| \033[1;91m | Red            |
| \033[1;92m | Green          |
| \033[1;93m | Yellow         |
| \033[1;94m | Blue           |
| \033[1;95m | Purple         |
| \033[1;96m | Cyan           |
| \033[1;97m | White          |

After running the antlr4 tool on the weird calc grammar, you will notice a file is created with the extension .tokens. This file contains the tokens and token types.

Once this is working, you should have syntax highlighted text!

```

$ java Highlight source
// from homework
print($2 ~ -3 @ -1 # 3); // should print 1
print(--5 @ $4);         // should show: line 3:7 no viable alternative at input '--'
                        // may print a 0 after the error (if so, ignore it)
print(34 ~ $8);           // should print 1157
print(8 - 4 # 4 # 5);     // should show: line 5:8 mismatched input '-' expecting {'}', '#'}
print($-3 @ 0 # $1);     // should print 0
print();

// simple variable test
counter = 0;             // should assign 0 to counter
print(counter);          // should print 0
counter = counter @ 1;   // should assign 1 to counter
print(counter);          // should print 1
print();

// more complicated variable test
number = -5;             // should assign -5 to number
print(number);           // should print -5
print($number);          // should print -1
number = number ~ number; // should assign 50 to number
print(number);           // should print 50
print(number # 5);        // should print 0
print(number # 4);        // should print 1
print(number @ -125);     // should print 75

```

Now let's add a way to allow the user to set their color profile. We will use JSON to specify desired colors. Instead of specifying them for numeric token types (which isn't very user friendly), we will use made up names for groups of tokens. Create two files called "theme1.json" and "theme2.json". Here are the contents of the files:

| theme1.json                                                                                                                                                  | theme2.json                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>{   "default" : "white",   "keyword" : "blue",   "math_operator" : "yellow",   "number" : "red",   "comment" : "green",   "identifier" : "cyan" }</pre> | <pre>{   "default" : "white",   "keyword" : "red",   "math_operator" : "cyan",   "number" : "purple",   "comment" : "green",   "identifier" : "yellow" }</pre> |

You should allow the user to specify which color theme they want as an additional command line argument (without the file extension). For example, running the program like this:

```
java Highlight source theme1
```

Will use theme1.json.

Here are the mappings from token group names to tokens:

| Color theme group name | Token(s)                                   |
|------------------------|--------------------------------------------|
| default                | semicolon, equals, left paren, right paren |
| keyword                | print                                      |
| math_operator          | # @ ~ \$ -                                 |
| number                 | digits                                     |
| comment                | line comment                               |
| identifier             | identifier                                 |

Now, we *could* use a Java library to deserialize the JSON into a custom class object. But this is probably overkill for something so simple. Plus, we need more practice with ANTLR! So, I want you to create a grammar file (in addition to the WeirdCalc.g4 file) that will parse JSON. Call it "Json.g4". Now, it only has to parse our color theme files! This means plenty of features that JSON supports does NOT need to be in our grammar. Keep it simple. It's actually very easy to create a simple grammar file to parse just the JSON above. In fact, any extra features added to your grammar file will result in point penalties. As you parse the file, you should be loading the key, value pairs you find into a HashMap in your Highlight java class.

Finally, let's put this all together. In Highlight.java you should create a lexer and parser object for your json grammar (after running antlr4 on your json grammar) and run it for the given theme file (whose name is taken from command line arguments). Then take your source wc file (whose file name also comes from command line args) and run the WeirdCalc lexer over it. As you print out the tokens, use the color theme map to switch colors appropriately. Test with both themes to make sure you did everything right.

### For submission:

If NOT doing the bonus:

Only submit the single grammar file WeirdCalc.g4. I will use the ANTLR tool to generate the code and run it with my included WeirdCalc.java file and use my included source.wc as the input. I will also use an input file that tests for error handling when using a variable that hasn't been created or initialized. This means that all work for this assignment needs to be done in the grammar file, as any alterations to the WeirdCalc.java file or the generated files will not be used when I test this. Submit the grammar file to Canvas.

If doing the bonus:

Create a folder. In that folder put your WeirdCalc.g4 file from the normal assignment. Also in that folder, put a folder named “bonus”. Inside that bonus folder, put the bonus version of your WeirdCalc.g4 file along with your Json.g4 and Highlight.java files (so 3 files in total for the bonus).

I will test your normal assignment in addition to your bonus. When I test your bonus, I will add theme1.json and theme2.json (with contents EXACTLY as it shows above) and I will use the given source.wc file.

Take the root folder (with WeirdCalc.g4 and bonus subfolder) and zip it up. Submit this zip.

**Rubric:**

| #            | ITEM                        | POINTS          |
|--------------|-----------------------------|-----------------|
| 1            | Grammar – Lexer             | 10              |
| 2            | Grammar – Parser            | 20              |
| 3            | Error reporting             | 10              |
| 4            | Correct output of source.wc | 20              |
| 5            | Bonus                       | +10             |
| <b>TOTAL</b> |                             | <b>60 (+10)</b> |

**Penalties**

| # | PENALTIES                                       | POINTS |
|---|-------------------------------------------------|--------|
| 1 | Doesn't compile                                 | -50%   |
| 2 | Doesn't execute once compiled (i.e. it crashes) | -25%   |
| 3 | Late up to 1 day                                | -25%   |
| 4 | Late up to 2 days                               | -50%   |
| 5 | Late after 2 days                               | -100%  |